Studio Nx

Backend(FastAPI/Python) 41 — GET · PATCH /api/v1/users/me

2026-10-07

Backend(FastAPI/Python) 41 — `GET · PATCH /api/v1/users/me`

항목	내용
파일	`studionx/backend/app/api/v1/users.py` — `read_me()` · `update_me()`
작업	P1-08 회원 정보 — 내 정보 조회 · 닉네임 변경 · PRD 01 FR-106
작성	Claude (AI) · 2026-10-07 · 사용자 검토
이전 · 다음	40 `profile_image_file()` · 42 `POST · DELETE /users/me/profile-image`

1. 하는 일

요청	응답
`GET /api/v1/users/me` + `Authorization: Bearer <Access>`	200 내 정보 7칸 (36)
`PATCH /api/v1/users/me` + `{"nickname": " Route_New "}`	200 바뀐 내 정보 (`"nickname": "Route_New"`)
`PATCH` + `{}`	200 그대로
`PATCH` + `{"email": "new@example.com"}`	200 그대로 — 이메일은 바꿀 수 없음 (FR-106), 모르는 칸은 무시
`PATCH` + `{"nickname": " a "}` · 21자	422 `VALIDATION_ERROR`, `details.nickname`
`PATCH` + 남이 쓰는 닉네임	409 `NICKNAME_TAKEN` · 탈퇴 3일 안이면 409 `REJOIN_TOO_SOON` (37)
토큰 없음 · 틀림 · 탈퇴 회원	401 `UNAUTHORIZED` (06 `CurrentUser`)

2. 언제 쓰나

화면 (JS)	부르는 것
userInfo.html `loadMe` (P1-10)	`GET` — 닉네임 · 이메일 · 사진 칸 채우기
userInfo.html 닉네임 Save changes (P1-10)	`PATCH`
myPage.html (P1-17)	대시보드 API(P1-16)가 따로 있음 — 이 API 는 회원 정보 화면용

함수들이 이어지는 모습 (`PATCH`)

순서	하는 곳	일
①	06 `CurrentUser`	토큰 → 활동 회원 (아니면 401)
②	`MeUpdateRequest` (37)	본문 검사 — 공백 제거 후 2~20자 (아니면 422, 03 처리기)
③	37 `update_me`	중복 · 탈퇴 3일 확인 → 저장 (아니면 409)
④	36 `to_me_response`	7칸 응답

3. 코드

import (파일 맨 위)

```
from fastapi import APIRouter, Response, UploadFile, status
from fastapi.responses import FileResponse

from app.api.deps import CurrentUser, Db
from app.schemas.common import ErrorResponse
from app.schemas.user import (
    MeResponse,
    MeUpdateRequest,
    PasswordChangeRequest,
    WithdrawRequest,
)
from app.services import user_service
```

`user_service` 줄을 이번에 추가했다.

함수 (경로 · 응답 설정은 템플릿 그대로)

```
@router.get("/me", response_model=MeResponse, responses={401: {"model": ErrorResponse}})
def read_me(user: CurrentUser) -> MeResponse:
    return user_service.to_me_response(user)


@router.patch(
    "/me",
    response_model=MeResponse,
    responses={409: {"model": ErrorResponse}, 422: {"model": ErrorResponse}},
)
def update_me(data: MeUpdateRequest, user: CurrentUser, db: Db) -> MeResponse:
    return user_service.to_me_response(user_service.update_me(db, user, data))
```

문법 풀이

import 문법	뜻
`from app.services import user_service`	모듈 통째로 — `user_service.update_me` 처럼 **어느 파일의 함수인지** 보이게 (라우터 함수 이름도 `update_me` 라 겹치지 않음)

식	뜻
`@router.get("/me", …)` · `@router.patch(…)`	이 라우터는 `/api/v1/users` 에 붙어 있음 → `GET · PATCH /api/v1/users/me`
`user: CurrentUser`	06 의 의존성 — 토큰을 검사해 회원을 넣어 줌
`data: MeUpdateRequest`	JSON 본문을 이 모양으로 검사해서 넣어 줌. 틀리면 함수가 불리지도 않고 422
`db: Db`	요청마다 DB 세션 (P1-03)
`response_model=MeResponse`	응답을 이 모양으로 한 번 더 걸러 보냄 · 문서(`/docs`)에 표시

4. 흐름

GET: 토큰 확인 → 36 으로 응답. **PATCH:** 토큰 확인 → 본문 검사 → 37 로 바꾸기 → 36 으로 응답.

5. 왜 이렇게

선택	이유
라우터는 한 줄 (일은 서비스에)	P1-04 · 05 와 같은 원칙 — 규칙은 서비스에서 시험하고, 라우터는 연결만
모르는 칸(`email`)은 무시	pydantic 기본 동작. 이메일을 바꾸려는 요청도 오류 없이 "안 바뀜" 으로 끝난다 — 화면은 이메일 칸을 읽기 전용으로 보여 줌
바뀐 정보를 응답으로	화면이 다시 `GET` 하지 않아도 된다
`user_id` 가 응답에 있어도 됨	토큰 주인 본인에게만 감 (36 §2)

6. 시험 (`tests/test_user_routes.py` 에 9개 추가 · 모두 11개, 실제 앱 + MySQL)

시험	확인한 것
`GET`	200, 7칸, 값, 사진 `null`, 응답에 비밀번호 해시 글자 없음
`GET` 토큰 없음	401 `UNAUTHORIZED`
`PATCH` 새 닉네임	200 `Route_New`, DB 도
`PATCH {}`	200 그대로
`PATCH` 이메일	200, 이메일 그대로
`PATCH " a "` · 21자	422 `VALIDATION_ERROR`, `details` 에 `nickname`
`PATCH` 남의 닉네임 (대소문자만 다름)	409 `NICKNAME_TAKEN`
`PATCH` 토큰 없음	401

전체 시험 299개 통과, `ruff` 검사 통과.